

TAREA #1 – MICROPROCESADORES Y MICROCONTROLADORES

Preguntas teóricas:

1. Explique la principal utilidad de Git como herramienta de desarrollo de código.

La principal utilidad de Git como herramienta de desarrollo es proporcionar un sistema de control de versiones distribuido que permite mantener la integridad, el rastreo histórico y la colaboración dentro de un proyecto de software. Su propósito fundamental es registrar de forma organizada los cambios realizados en el código, permitiendo recuperar versiones anteriores, comparar modificaciones y desarrollar nuevas funcionalidades sin comprometer directamente una versión estable del proyecto.

Git es una de las herramientas fundamentales de la ingeniería de software moderna. Su verdadera utilidad radica en coordinar el trabajo de múltiples personas sobre los mismos archivos; mediante mecanismos de ramificación y fusión, Git permite integrar aportaciones individuales cuando los cambios son compatibles y detectar conflictos cuando existen modificaciones que no pueden combinarse automáticamente. De esta manera, ofrece un registro transparente de quién realizó cambios, cuándo se realizaron y cómo ha evolucionado el proyecto.

2. ¿Qué es un commit?

En el desarrollo de software, un commit representa la unidad fundamental de cambio guardada en el historial de un proyecto. Lejos de ser un simple guardado automático, es una acción deliberada mediante la cual un desarrollador consolida un conjunto de modificaciones específicas y las registra de forma permanente en un sistema de control de versiones.

El commit es el bloque de construcción de la colaboración tecnológica. No solo protege el trabajo individual contra pérdidas accidentales, sino que comunica al resto del equipo la evolución y el propósito de cada modificación en el sistema, permitiendo que decenas de programadores reconstruyan, auditen o integren soluciones sobre una misma base de código sin interferir entre sí.

3. ¿Qué es un Branch?

En el desarrollo de software, un Branch (o rama) es una línea de tiempo alternativa y paralela dentro del historial de un proyecto. Permite a los desarrolladores desvincularse temporalmente del tronco principal de trabajo para experimentar, corregir errores o diseñar nuevas funciones con total libertad y sin alterar el código que ya funciona.

El Branch es la herramienta que hace posible el desarrollo simultáneo a gran escala. Actúa como un espacio de trabajo personal o temático donde las ideas se prueban en aislamiento; una vez que la propuesta es estable, se fusiona limpiamente con el proyecto central, permitiendo que un equipo avance en múltiples direcciones a la vez sin generar caos.

4. En el contexto de GitHub, ¿Qué es un Pull Request?

En el contexto de GitHub, un Pull Request (o PR) es una propuesta formal de colaboración mediante la cual un desarrollador solicita la incorporación de sus cambios a la línea de tiempo principal de un proyecto. Lejos de ser una simple transferencia de archivos, representa un espacio de diálogo, auditoría y consenso donde un equipo valida que el nuevo código cumpla con los estándares de calidad antes de fusionarlo de manera definitiva.

El Pull Request es el corazón de la cultura de código abierto y del trabajo en equipo moderno. Transforma el acto de programar de un esfuerzo solitario a un proceso comunitario transparente; garantiza que ninguna modificación se implemente a ciegas y fomenta el aprendizaje colectivo, asegurando que la base de datos central permanezca siempre estable, limpia y funcional.

5. ¿Si un usuario quiere “Actualizar se repositorio contra el Branch master” que debe de hacer?

Cuando un usuario necesita actualizar su repositorio local con los cambios más recientes del Branch master remoto, debe realizar un proceso de sincronización e integración. Si se encuentra trabajando directamente sobre master, puede utilizar el comando `git pull origin master`, el cual obtiene los cambios disponibles en el repositorio remoto y los integra en la rama local. También es posible realizar el proceso por separado utilizando `git fetch` para descargar los cambios y posteriormente integrarlos mediante `git merge`.

Mantener el repositorio actualizado es una práctica preventiva importante en el desarrollo en equipo. Evita que un programador continúe trabajando sobre una versión desactualizada del software, disminuye la posibilidad de generar conflictos posteriores y permite que cualquier nueva modificación se realice tomando como base los cambios más recientes que han sido incorporados al Branch master del proyecto.

6. ¿Bajo qué condiciones una herramienta como Git necesita apoyo de un humano para saber cómo integrar cambios locales con cambios remotos?

Una herramienta como Git necesita el apoyo de un humano principalmente cuando se produce un conflicto de fusión o merge conflict. Esta situación puede ocurrir cuando los cambios locales y remotos modifican de manera incompatible una misma parte de un archivo y Git no puede determinar automáticamente cuál de las modificaciones debe conservarse o cómo deben combinarse.

En estas condiciones, Git detiene la integración y señala dentro de los archivos las zonas donde existe el conflicto. El sistema cede la decisión al desarrollador porque no posee el contexto necesario para determinar cuál modificación representa el comportamiento deseado del programa; por esta razón, una persona debe revisar las versiones en conflicto, decidir cuál conservar o cómo combinarlas y posteriormente confirmar la resolución mediante un nuevo commit.

7. ¿Qué es una Prueba Unitaria o Unittest en el contexto de desarrollo de software?

En el contexto del desarrollo de software, una Prueba Unitaria (o Unittest) es un método automatizado diseñado para verificar el comportamiento de la pieza de código más pequeña y aislada de una aplicación, como una función, un método o una clase. Su objetivo es aislar esta sección lógica del resto del sistema para demostrar rigurosamente que produce el resultado correcto ante un estímulo específico.

Las pruebas unitarias actúan como la red de seguridad del código y el pilar de metodologías como la integración continua. Al documentar matemáticamente cómo debe comportarse cada fragmento del software, permiten que cualquier desarrollador realice cambios, refactorice o actualice el sistema con la absoluta confianza de que no dañará las funcionalidades existentes, reduciendo drásticamente los costos de mantenimiento y acelerando el ritmo de entrega del equipo.

8. Bajo el contexto de pytest. ¿Cuál es la utilidad de un “assert”?

En el contexto de pytest, la palabra clave `assert` es el mecanismo fundamental de validación y el veredicto final de cualquier prueba unitaria. Su utilidad principal es evaluar si una condición o resultado matemático del código es verdadero (True) o falso (False), actuando como el juez que determina de forma automatizada el éxito o el fracaso de un test.

El `assert` es el punto donde la teoría del desarrollo se encuentra con la realidad del código. En equipos de trabajo, proporciona una documentación viva y matemática del comportamiento esperado del software; cuando una modificación rompe una regla del negocio, el `assert` actúa como una alarma instantánea en los sistemas de integración continua, impidiendo que el código defectuoso avance hacia producción y garantizando la estabilidad del producto final.

9. ¿Qué es Flake8?

En el contexto del desarrollo de software en Python, Flake8 es una herramienta de análisis estático de código, comúnmente denominada linter. Su utilidad principal es analizar los archivos de código fuente sin necesidad de ejecutar el programa, con el objetivo de detectar problemas relacionados con el estilo de programación, errores potenciales y ciertas prácticas que pueden afectar la calidad y legibilidad del código.

Flake8 actúa como una herramienta de control de calidad dentro de un proyecto de desarrollo. Entre otras funciones, puede detectar incumplimientos de las convenciones de estilo de Python, importaciones innecesarias, variables no utilizadas y problemas relacionados con la complejidad del código. Su utilización ayuda a mantener un estándar de programación uniforme, facilita las revisiones de código y permite corregir problemas antes de que el software sea integrado o desplegado.

10. Comente sobre la utilidad de la aleatorización en pruebas de código.

En el contexto de la ingeniería de software, la aleatorización en las pruebas de código es una técnica utilizada para descubrir errores ocultos, comportamientos inesperados y fallos asociados a condiciones límite (edge cases). Su utilidad principal radica en generar diferentes datos de entrada de manera aleatoria, permitiendo evaluar el funcionamiento del programa ante escenarios que el desarrollador posiblemente no habría considerado mediante pruebas diseñadas únicamente de forma manual.

La aleatorización aporta una mayor variedad de escenarios de prueba y ayuda a detectar supuestos incorrectos presentes en el código. Sin embargo, para que este tipo de pruebas sea realmente útil, es importante poder reproducir los casos que producen un fallo, por ejemplo, conservando o fijando la semilla utilizada por el generador aleatorio. De esta forma, una prueba aleatoria no solo permite descubrir un comportamiento defectuoso, sino también repetir exactamente las condiciones que lo provocaron para analizarlo y corregirlo.